flask 애플리케이션 작성 및 컨테이너화

업무 영역	Devops 자동화
기능 유형	문서 작업
진행현황	작업완료
담당자	동욱 김
시작일	@2025년 8월 12일
진행률	1
설명	Flask 앱을 Docker로 감싸서 실행 가능한 환경 구성 -[O] Dockerfile 작성 (Flask 앱 실행 환경 설정) -[O] Flask 앱을 Docker로 감싸서 실행 가능한 환경 구성
주차	1~2주차

목표: Flask 애플리케이션을 Docker 컨테이너로 실행하고, 포트 바인딩 및 UFW(방화벽) 연동 확인

2. Dockerfile 작성

목적: Flask 앱 실행 환경 설정

- 환경 설치를 한곳에 모아둔 requirements.txt 작성

```
Flask==3.0.3
gunicorn==23.0.0
psutil==6.0.0
```

- Dockerfile 작성

```
FROM python:3.12-slim
```

```
# 보안/재현성에 유리한 기본 설정
ENV PYTHONUNBUFFERED=1 \
    PORT=5000 \
    GUNICORN_WORKERS=2 \
    GUNICORN_TIMEOUT=30

WORKDIR /app

# 의존성 먼저 설치 (빌드 캐시 최적화)
COPY requirements.txt .
RUN pip install --upgrade pip \
 && pip install --no-cache-dir -r requirements.txt \
 && useradd -m appuser

# 소스 복사
COPY . .

# 컨테이너 내부 헬스체크(옵션)
# curl 설치(슬림 이미지에는 기본 미포함)
RUN apt-get update && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -fsS http://127.0.0.1:${PORT}/health || exit 1

USER appuser

EXPOSE 5000

# 워커 2개로 gunicorn 실행 (app:app → 파일명:변수명)
CMD ["sh", "-c", "gunicorn -w ${GUNICORN_WORKERS} -t ${GUNICORN_T
IMEOUT} -b 0.0.0.0:${PORT} app:app"]
```

- 빌드 속도 증가, 이미지 용량 감소를 위한 .dockerignore 파일 작성

```
# Byte-compiled / optimized / DLL files
__pycache__/
```

```
*.py[codz]

# logs
*.log

# Environments
.env
.venv
venv/

# git files
.git
.gitignore

# docker files
Dockerfile*
docker-compose*
```

로컬 환경에서 커밋, pull을 실행하고 ec2에서 push 실행

EC2에 Docker 환경 구성

업무 영역	Devops 자동화
진행현황	작업완료
담당자	동욱 김
진행률	1
설명	Docker, Docker Compose 설치 -[O] EC2에 Docker 설치 (sudo apt install docker.io) -[O] Docker Compose 설치 (추후 자동화 대비) -[O] Elastic IP 할당 및 연결 (필요 시)
주차	2주차

1. Docker Compose 설치 (추후 자동화 대비)

- docker compose 설치

```
sudo apt install -y docker-compose

docker-compose --version

# v2가 설치되지 않아서 v1으로 설치
```

- app.py, dockerfile과 같은 폴더에 docker-compose.yml 작성

```
version: "3.9"
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    image: autoscaling-flask:latest
    container_name: autoscaling-web
    restart: always
```

```
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - GUNICORN_WORKERS=2
      - GUNICORN_TIMEOUT=30
    # env_file:
    #   - .env
```

- docker compose 실행 및 확인

```
docker compose up -d

#퍼블릭IP:5000에서 확인
```

2. Elastic IP(EIP) 할당 및 연결

- **EC2 콘솔 진입** → 왼쪽 메뉴 **Elastic IPs**
- **Allocate Elastic IP address** 클릭
  - *Amazon's pool of IPv4 addresses* 선택 → 할당
- 목록에서 방금 만든 EIP 선택 → **Actions → Associate Elastic IP address**
  - **Resource type: Instance**
  - **Instance**: 현재 EC2 선택
  - **Private IP**: 기본(Primary) 선택
  - (기존 퍼블릭 IP가 있었다면) **Allow reassociation** 체크
  - **Associate** 클릭
- **확인**
  - 인스턴스 상세 화면에 **Public IPv4 address가 EIP로 바뀐 것** 확인
  - 보안 그룹/ufw에서 **TCP 5000 허용**되어 있는지 확인
  - 브라우저에서 `http://<EIP>:5000` 접속 테스트